AI Engineering and Claude Architect · Part IV, Advanced

Advanced AI Topics, Lesson 5

Event-Driven AI, Workflow Engines and Sagas

The worker crashed after the charge, and before saving that it had. It restarted, and asked the model again, which now gave a different answer. Durable systems assume repeatable steps, and a model isn't one.

Module	AI11 of 13
Lesson	5 of 5
Prerequisites	HLD M08 and M11 (messaging, sagas), AI06 L7, AI07 L2, AI08 L3 and L6
Exam relevance	Engines aren't in the guide. Supports TS 1.1, 1.4, 1.6, 4.5
Reading time	~35 minutes

Contents

1. Event-Driven AI, Workflow Engines and Sagas
2. Module Revision

Lesson 5 of 5 · Advanced AI Topics

Event-Driven AI, Workflow Engines and Sagas

Key idea

Record each model call as a completed step, so that recovery replays its answer and doesn't ask again, and give every side effect an idempotency key. A durable history stops most repeats, and keys stop the rest. Sagas undo what can be undone, and some actions can't be, so put those last or behind a person.

> *Before: HLD's messaging and sagas, the agent loop, and Java async. Teaches: why AI changes durable execution, event triggers, crash-testing a workflow, replay and model calls, sagas with irreversible steps, and human waits. Used by: AI12, AI13.*

The hook: every crash point

A small order flow has four steps: a model decides, then charge, ship, and email. Failures can occur before each step takes effect, or after it takes effect and before that's saved. So I crashed it once at **each of the 8 points**, restarted it, and counted what the outside world ended up with. **This is a tiny model of a durable engine that I wrote for teaching.** Real output from `SagaLab.java`:

```
One crash at each of 8 points, then a restart:
  design                              2xcharge  2xship 2xemail  correct
  restart from the top                       5       3       1     3/8
  restart, with idempotency keys             0       0       0     8/8
  durable history, no keys                   1       1       1     5/8
  durable history, with keys                 0       0       0     8/8
```

Three readings:

→ **Restarting from the top double-charges in 5 of 8 cases.** That's what a retried job or a redelivered message does by default.

→ **Durable history helps, and it isn't enough on its own.** Row three still double-charges once. The crash came after the effect and before the record, and no recording system can close that gap by itself.

→ **Idempotency keys close it.** The outside system recognizes the repeat.

Durable execution reduces repeats, and only the receiver can make a repeat harmless. So why is row two, keys alone, not the end of the story? Because of the model.

Intuition: the recipe and the notebook

A cook follows a recipe, ticking steps off in a notebook. If the power goes out, they read the notebook and carry on. Workflow engines do just that.

Now suppose one step is "ask the head chef what to make." Ask twice, and you may get a different answer, while half of the first dish is already in the oven. So the rule is to write down the chef's answer the first time, and read it from the notebook after any interruption. You never ask again.

Mechanism

Replay, and why model calls are special

The Temporal docs describe the usual design. Workflow code doesn't act directly. Work that touches the outside world goes in **Activities**, and the results land in a durable **Event History**. After a crash, a new worker replays the history to rebuild its state. So, as the docs put it, "A Workflow Definition must be deterministic because Temporal reruns it to rebuild state after a failure."

A model call isn't deterministic (AI02 L5). It must be an Activity, so that its answer is recorded. Here's the difference, using a stand-in model that says "approve" 70% of the time:

```
Replaying a run after a crash, with a model that varies, 10,000 times:
  model answer not recorded: decision changed 4234 times
  model answer recorded:     decision changed 0 times
```

Without a recording, the decision changes 42% of the time on recovery. That matters because a run may have already charged the card based on the first answer, and now it sets off down a different path. That's why keys alone aren't enough, as row two would re-ask the model. It also pays for the call twice.

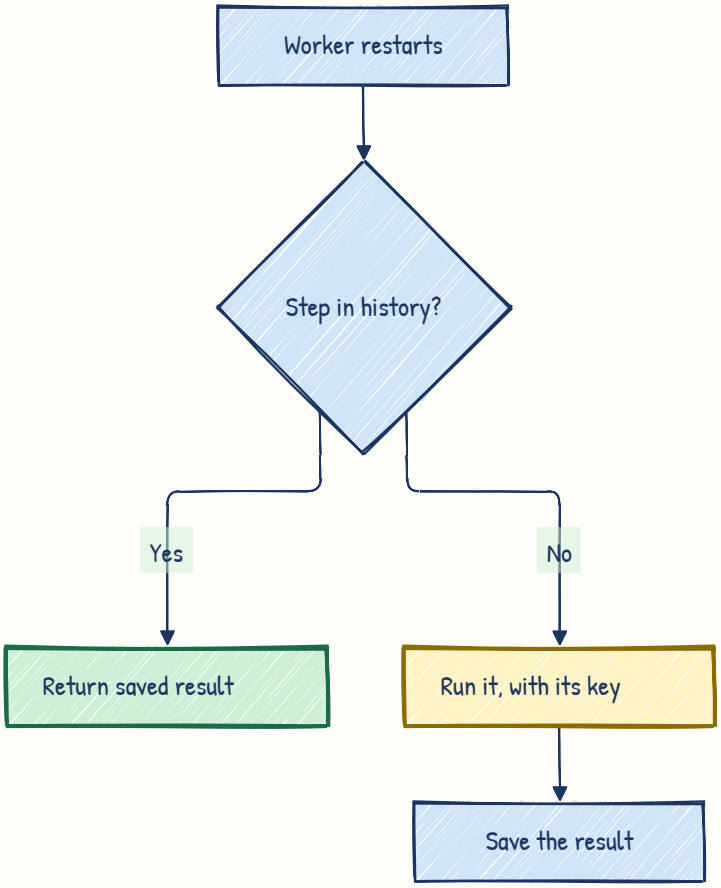


Figure 1. On recovery, recorded steps return their saved results without running. Only unfinished work actually executes.

What else in an AI workflow needs pinning

Replay needs the same inputs. Here's what can change between runs:

Thing	Why it matters	Rule
Model answer	Varies	Record it, as an Activity
Prompt version	A release may change while a run waits	Pin it in the workflow's state
Model ID	Retirement can pass during a long run	Pin it, and plan for expiry (AI10 L6)
Retrieved documents	The index changes	Record what was retrieved
Current time, random values	Differ on replay	Use the engine's versions
Tool results	Stale over long waits	Refresh, or tell the model (TS 1.7)

Long waits break other assumptions. A human approval can take three days. By then, the prompt cache has expired, so the next call costs more. Tool results may be stale, and a stored history must stay valid: tool results stay matched to their calls (AI06 L2), and thinking blocks need returning unchanged in tool loops, as the API's errors page describes.

Where events come from

AI08 L6 chose between sync, Kafka, Batch and engines. Event-driven AI starts work without a person waiting.

Trigger	Example	Note
Domain event	Order placed, ticket opened	Kafka or similar, with at-least-once delivery
Webhook	Pull request opened	Verify the signature
Schedule	Nightly triage	Managed Agents has scheduled deployments
Batch	Monthly summaries	Batch API at half price (TS 4.5)

Duplicates are normal. At-least-once delivery repeats messages, and each repeat can buy another model call. An invented 2% duplicate rate, at $0.013 per call:

```
At-least-once delivery:
  100,000 events, delivered 101,968 times at a 2% duplicate rate
  no dedupe: 101,968 model calls. With a seen-ID check: 100,000. Waste avoided at $0.013 each: $26
```

$26 isn't the main concern. A duplicate ticket reply is. Check event IDs before starting work.

Fan-out can surprise you. One event can start many agents, and a burst of events can hit your rate limits (AI10 L1). So put a queue and a concurrency limit between events and the model. AI08 L6's worker pattern does this.

Managed Agents options. The API reference describes Anthropic's versions. Scheduled deployments create a session on a cron schedule, and a rate-limited firing is recorded with **no retry**, so the schedule tries again at its next time. Webhooks are thin: they carry the event type and resource IDs, and you fetch the resource for its current state. That suits duplicates and ordering, because you always read the latest.

Shape: chain, workflow, or agent?

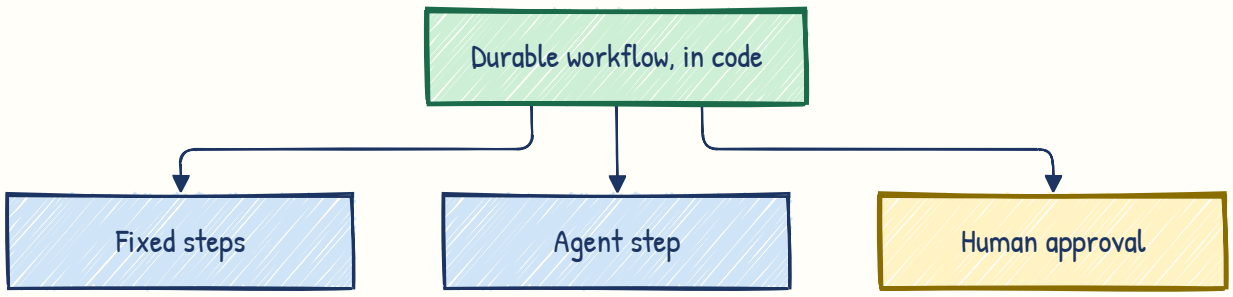


Figure 2. Use code for what's known, and an agent for what's open-ended. An agent can sit inside a durable workflow, as one of its steps.

Choice	When	Exam term
Prompt chain in code	Predictable steps	TS 1.6
Agent loop	Open-ended investigation	TS 1.1, TS 1.6
Workflow around both	Long waits, retries or several systems	Not in the guide

Granularity. Should an agent's whole run be one Activity, or each turn? As one Activity, a crash loses and repays the whole run. Per turn, each model call and tool call is recorded, and recovery resumes exactly, at the price of more events and engineering. The per-turn form is what AI08 L3's ledger does in small. For short agents, a whole-run Activity with safe retry can be enough. For long ones, record each turn.

Sagas

HLD M11 taught sagas: a long transaction is a series of local steps, each with a compensating action to run on failure. Real output:

```
Saga, when a step fails:
  [reserve stock, charge card FAILED, undo: release stock]
  [reserve stock, charge card, send email, ship parcel FAILED, undo: send a correction (the email can't be unsent),
undo: refund card, undo: release stock]
```

The second line is the AI-relevant one. **A sent email can't be unsent, and a compensation may be only an apology.** Agents make this worse, as they tend to act on the world in new ways. Your rules:

Rule	Why
Order steps with irreversible ones last	Fewer failures after them
Require approval just before those	L4's reversibility
Make compensations idempotent	They're retried as well
Compensate from code	A model shouldn't improvise its undo
Record what happened	Audit, and a person may have to finish
Allow "stuck, needs a person" as an outcome	Some failures have no automatic undo

Human steps as signals

Temporal's docs describe Signals, which a running workflow can receive. The workflow simply waits, with no thread or model call held open.

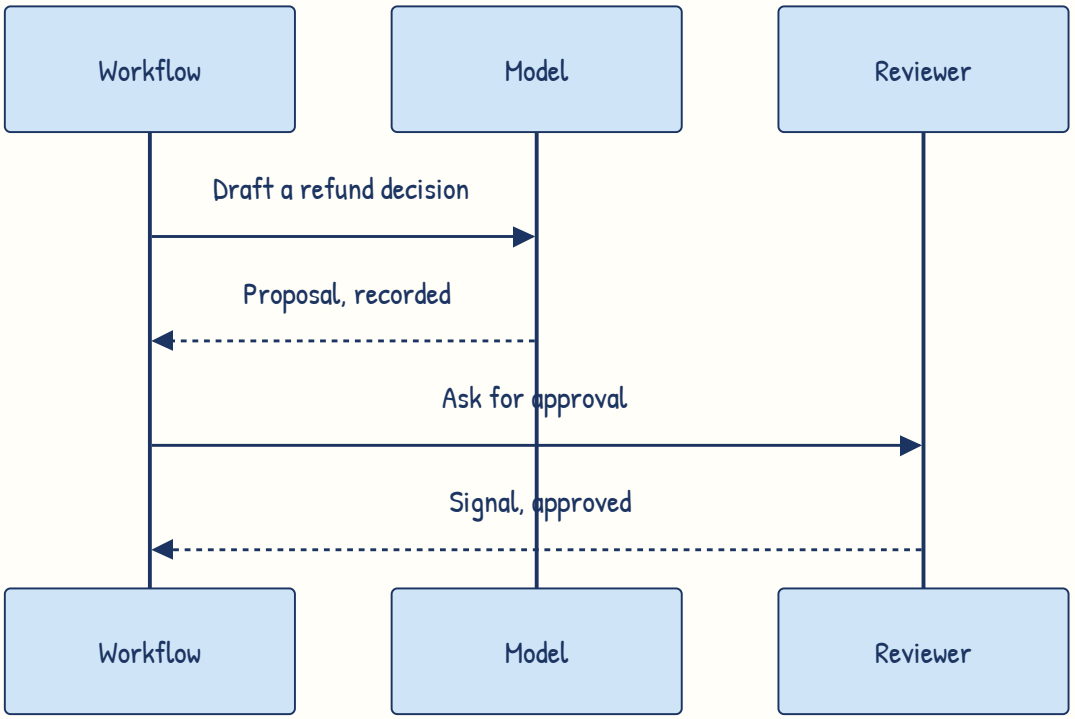


Figure 3. The workflow pauses durably. It survives restarts during the wait, and a timeout means the task doesn't stall forever.

Hand over a structured summary (TS 1.4 and TS 5.2): customer, root cause, amount, and recommended action, because the reviewer doesn't have the transcript in their head. Add a timeout and an escalation path.

Do you need an engine?

Situation	Choice
Minutes long, one service	AI08 L3's ledger, plus a queue
Hours long, some retries	Database state machine, or Managed Agents
Days long, human waits, many systems	A workflow engine
Framework checkpointing	AI07 L2's LangGraph-style checkpointing, if already in use

An engine is a critical system to run, and developers must learn its rules on determinism. I haven't run Temporal, and my lab is a teaching model.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Retry the job	5 of 8 crash points double-charged
Durable history only	Crash between effect and record: 5 of 8 correct
Idempotency keys only	Model re-asked, and 42% changed in the lab
Model call in workflow code	Replay isn't deterministic
Ask the model to work out what's left after a crash	It can't see true state. Use records
A compensation for everything	Some actions can't be undone
Hold a thread during approval	Lost on restart

How Anthropic and the field approach it

Anthropic publishes agent loop guidance (AI06 L2), long-running harness patterns (AI06 L7), the Batch API, and Managed Agents with scheduled deployments, webhooks and durable sessions. I haven't seen a publication describing how its internal services use workflow engines, so I won't state any.

The field. Temporal's docs define the replay model used here. Sagas are long-established, and HLD M11 has them. AI07 L2 compared framework checkpointing.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a consumer that calls the model, then acts.

Improved: retries, and idempotency keys. New problem: a crash re-asks the model, long waits hold resources, and partial failures need undoing.

Production: a workflow with recorded steps, and a queue that protects your limits.

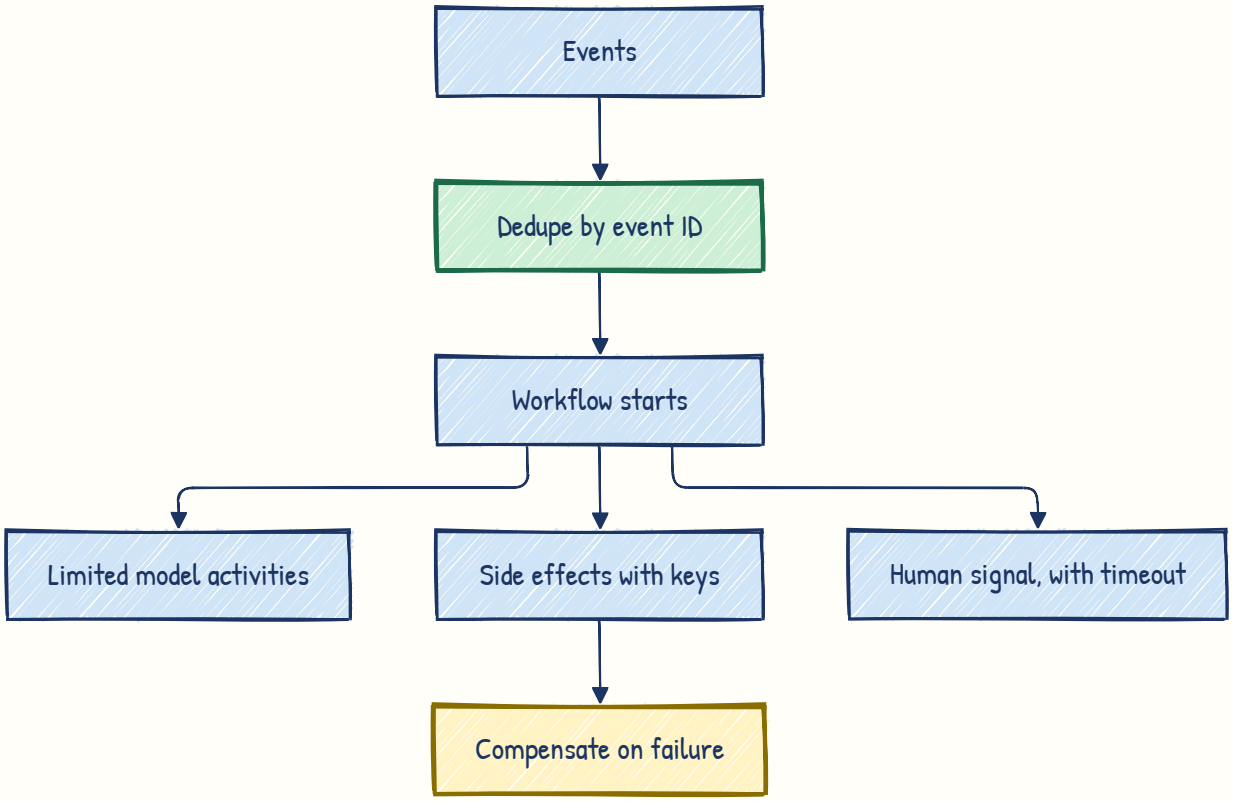


Figure 4. Events are deduplicated first. A limit protects the provider. Each step is recorded, and side effects carry keys.

Component	Why it exists	How it communicates
Event broker	Decouples producers	At-least-once delivery
Deduper	Stops repeats starting work	Store of seen IDs
Workflow engine	Durable state, timers and retries	Worker polling
Model activities	Recorded answers	Through the gateway (AI10 L7)
Concurrency limit	Protects rate limits	Task queue
Idempotent backends	The real guard against duplicates	Key on each write
Compensations	Undo in code	Saga steps
Reviewer queue	Human steps	Signals
Trace links	One trace per run across events	Trace ID in events (AI10 L4)

Failures:

Failure	How you'd notice	What to do
Crash after effect, before record	Duplicate rejected by key	Works as designed. Treat as success
Replay diverges	Engine's non-determinism error	Move the cause into an Activity
Prompt changes during a run	Odd behaviour	Pin version in the workflow
Event storm	Queue depth, 429s	Limit concurrency and shed
Poison message	Same failure repeating	Dead-letter queue (AI08 L6)
Compensation fails	Saga stuck	Retry, then a person
Human never replies	Timeout	Escalate, or a safe default
Model retires during a wait	404s on old runs	Allow an upgrade path, and test

Java

The engine's central rule is three lines: check the history, and skip the work if it's there.

```
if (history != null && history.containsKey(name)) {
    return (T) history.get(name);
}
```

The model call goes through the same method, so its answer is recorded like any other step:

```
String decision = engine.step("decide", model);
```

And the keys live in the outside system's code, as that's what actually stops the duplicate:

```
if (key == null || seenKeys.add("charge:" + key)) {
```

This is a teaching engine. Real ones add timers, versioning, workers and history limits. AI08 L6 has the runnable Kafka worker.

Trade-offs

Decision	Benefit	Cost
Workflow engine	Durable, with timers and retries	New critical system, and rules to learn
Ledger plus queue	Simple	You build timers and recovery
Per-turn recording	Exact recovery	More events and work
Whole-run Activity	Simple	Repeats and repays the run
Choreography	Loose coupling	Hard to see the flow (HLD M08)
Orchestration	One visible flow	A central dependency
Saga	Works across services	Partial states are visible

When not to: a short request-and-response agent needs AI08 L3's loop, a ledger and keys. Don't adopt an engine for work that finishes in a minute.

Common mistakes

Calling the model directly in deterministic workflow code. Replay either breaks, or quietly takes a different path. Make it an Activity.

Common mistakes

Believing "exactly once" came with the engine. A crash between effect and record repeats the effect. Keys on the receiver are what make this safe.

What to say in an interview

"I'd crash-test each step. A durable history stops most repeats, and the gap between effect and record needs idempotency keys at the receiver. Model calls must be recorded Activities, because replay isn't deterministic otherwise, and I'd pin prompt and model versions for long waits. Sagas undo what they can, with irreversible steps last and approved, and 'stuck, needs a person' as an allowed outcome. For short agents, I'd skip the engine."

Certification lens

How this lesson maps to CCAR-F

Workflow engines and sagas aren't in the guide. Nearby: prompt chaining versus dynamic decomposition (TS 1.6), handoff content (TS 1.4), Batch for work that can wait (TS 4.5), and resuming with stale results (TS 1.7).

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Developer Productivity · Medium. You paused a Claude Code session yesterday, and today several files it analyzed have changed. What's the best way to continue?

A. Resume, and assume its earlier analysis still holds

B. Resume, and tell it which files changed, or start fresh with a summary if much has changed

C. Delete the session history, and restart with no context

D. Copy yesterday's whole transcript into a new prompt

A **B** (TS 1.7). Resuming is useful when you inform it of the changes, and a fresh session with a summary suits heavily stale results. A acts on stale data. C loses useful context, and D carries over stale results, at a high token cost.

Q Q2 · Why did "durable history, no keys" still double-charge, and why doesn't "keys alone" solve everything?

A A crash after the charge and before the record leaves no sign that it happened, so replay charges again. Only the receiver can recognize a repeat. Keys alone protect effects, but restart re-asks the model, whose answer can change, as it did 42% of the time in my lab, sending the run down a different path. You need both.

Q Q3 · Design the order of steps for: send confirmation email, charge card, reserve stock, ask a model whether to flag for fraud.

A Model first, recorded, since it's free of side effects. Then reserve stock, which can be undone. Then charge, which can be refunded. Then the email last, as it can't be unsent. Each write carries a key. If the fraud flag requires review, add a human signal before charging.

Revision - Lesson 5

→ Crash-test each point. Restarting double-charged in 5 of 8 cases.

→ Durable history can't close the gap between effect and record. Keys at the receiver do.

→ Model calls must be recorded Activities. Unrecorded replay changed the decision 42% of the time.

→ Pin prompt and model versions, and record what was retrieved. Long waits expire caches.

→ Events arrive at least once, so dedupe first. Limit concurrency to protect rate limits.

→ Scheduled deployments don't retry a rate-limited firing. Webhooks are thin, so fetch current state.

→ Code for what's known, and an agent for the open-ended, often inside a workflow.

→ Sagas: irreversible steps last, compensate from code, and allow "needs a person."

→ Human steps are signals with timeouts, and structured handoffs.

→ An engine is for days-long work. Short agents only need a ledger and keys.

Your notes

Module Revision

Self-assessment

#	Can you...
1	Say where facts, rules, judgment and style each belong?
2	Climb the ladder before fine-tuning?
3	Give good reasons for fine-tuning?
4	Explain LoRA, and compute its parameter saving?
5	Explain why a small adapter can generalize better?
6	Explain why dataset size doesn't protect against poisoning?
7	List fine-tuning's later costs?
8	Rank reasons for self-hosting?
9	Size weights and KV cache memory?
10	Explain how memory limits batch size and cost?
11	Explain quantization, and how to test it?
12	Explain the GGUF memory trap?
13	Calculate cost by utilization, and find a break-even?
14	Say what you lose on leaving a provider?
15	Explain superposition and dictionary learning?
16	State Mapping the Mind's finding and limits?
17	State Tracing Thoughts' finding and limits?
18	Describe Sleeper Agents, with its caveat?
19	Describe Alignment Faking, with its caveat?
20	Describe Agentic Misalignment, with its caveat?
21	Turn these into engineering consequences?

#	Can you...
22	Compute compounding error?
23	Decide autonomy by reversibility and context?
24	Summarize what Anthropic measured on oversight?
25	Compare the code execution tool with your own sandbox?
26	Give the docs' computer use recommendations?
27	Explain why killing a process isn't containment?
28	Find the crash points in a workflow?
29	Explain what durable history and keys each fix?
30	Explain why model calls need recording?
31	List what to pin across long waits?
32	Order saga steps around irreversible actions?
33	Decide whether you need a workflow engine?

Where this module is used

Idea	Where it returns
Permissions, sandboxing, long sessions	AI12
Reversibility, gates and hooks on the exam	AI13
Session resume with stale results	AI12, AI13

Working space